

Lecture

Iteration: while-loop



Start

$t = 0$

amt = input
amount as
starting
balance

Is amt $\geq$
1000

No

$t = t + 1$
 $\text{amt} = \text{amt} * 1.2$

Yes

Output "It takes
 t years for your
investment to
grow to \$1000"

End

What kind of
loop is this?

$t:0$ amt: \$500

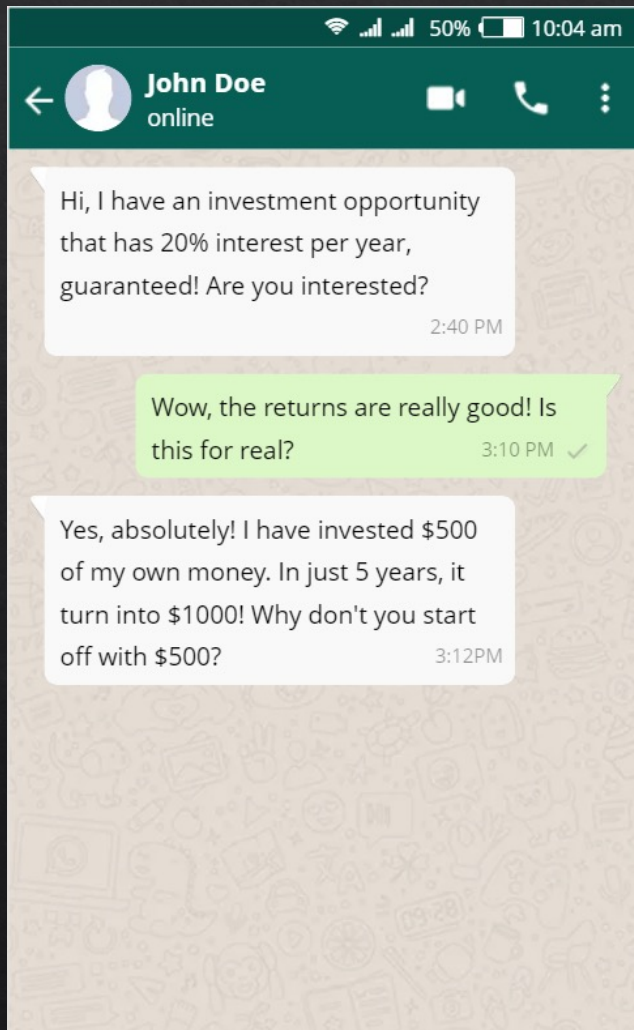
$t:1$ amt: \$600

$t:2$ amt: \$720

$t:3$ amt: \$864

$t:4$ amt: \$1036.80

It takes 4 years
for your
investment to
grow to \$1000.



while-loop is...

- is a control structure used to control the flow of execution.
- used to repeat a block of code while a condition has been met

```
while <condition>:  
    <code in the body of the loop>
```



while-loop example

Code:

```
total = 0

while total < 5:
    total = total + 1
    print(total)
print("Exit loop")
```

Output:

```
1
2
3
4
5
Exit loop
```

The body of code will loop until the expression `total < 5` is False, then we exit the loop.



if and while comparison

```
if <expression>:  
    <body>
```

```
while <expression>:  
    <body>
```

Body will be executed only
once if <expression> is
True

Body will be **repeatedly**
executed as long as
<expression> is True



Why while-loop?

Maximum 5 hobbies?



```
data = ""
for i in range(5):
    hobby = input("Hobby:")
    if hobby != "":
        data = data + ", " + hobby
    else:
        break
print("Your hobbies:", data)
```

How many
times do
we loop?



Why while-loop?

Instead of:

```
data = ""
for i in range(5):
    hobby = input("Hobby:")
    if hobby != "":
        data = data + ", " + hobby
    else:
        break
print("Your hobbies:", data)
```

We can code:

```
data = ""
hobby = input("Hobby:")
while hobby != "":
    data = data + ", " + hobby
    hobby = input("Hobby:")
print("Your hobbies:", data)
```

Not limited to a fixed number like 5



for-loop vs while-loop

for - loop:

- The process in the body repeated for **a fixed number of times.**

while - loop:

- The process in the body repeated while **a certain condition is true.**



Can we break or continue in a while-loop?



What does 'break' do?

Code:

```
total = 0
while total < 10:
    print(total)
    total = total + 1
    if total == 4:
        break
print("Exit loop")
```

Output:

```
0
1
2
3
Exit loop
```

break terminates the execution of the iteration, and exits the while-loop



What does 'continue' do?

Code:

```
total = 0
while total < 7:
    total = total + 1
    if total % 2 == 0:
        continue
    print(total)
print("Exit loop")
```

Output:

```
1
3
5
7
Exit loop
```

continue skips the current iteration
and continues the execution with the next iteration. 

Consider the following example:

```
value = int(input("Enter a positive integer value: "))  
  
while value != 0:  
    print(value)  
    value = value - 2  
  
print("Exit loop")
```

```
Enter a positive integer value: 10  
10  
8  
6  
4  
2  
Exit loop
```

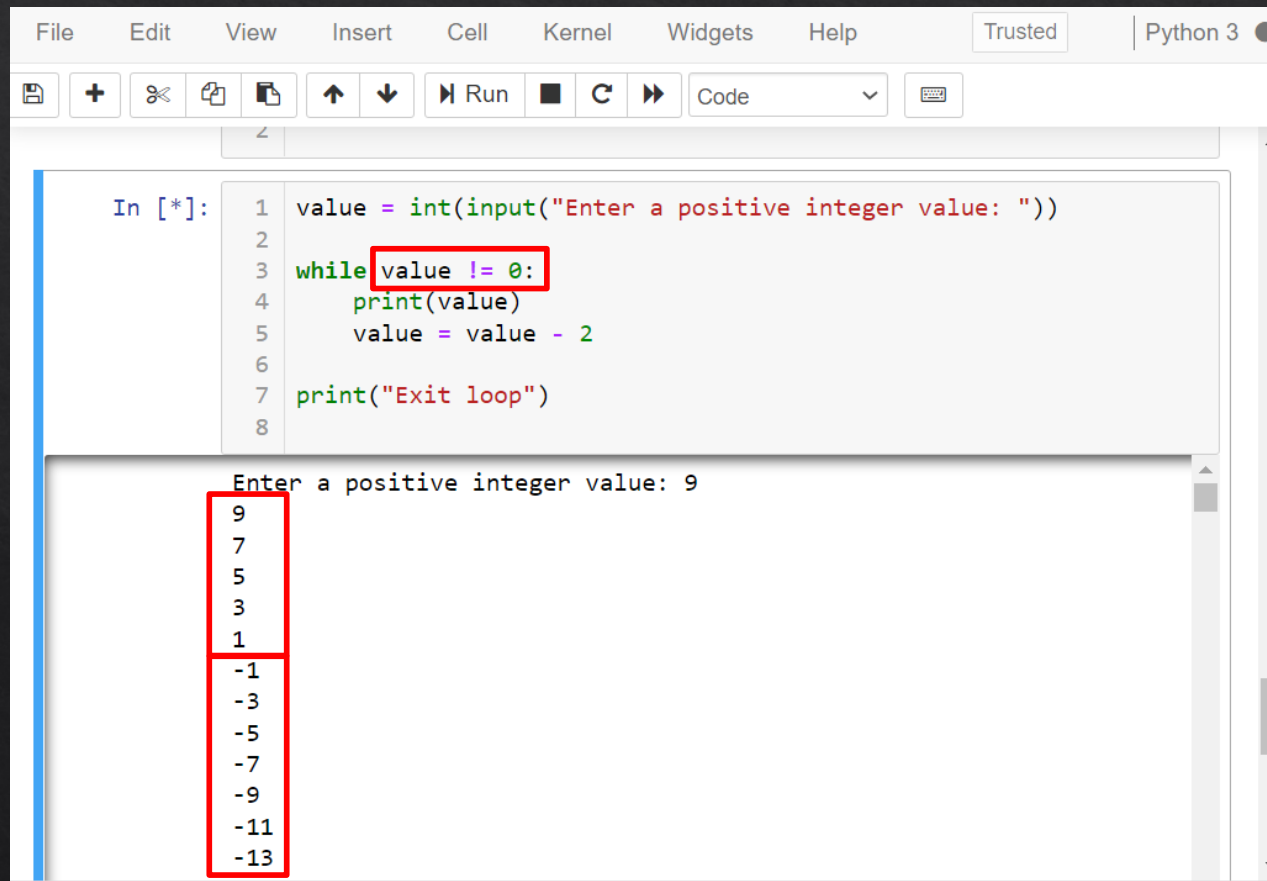
A student may enter a value 10 and it will reduce during the iteration until it becomes 0.

What will happen if the student enters a value 9?



What will happen if the student enters a value 9?

Stuck in Infinite Loop. Kernel is dead.



The screenshot shows a Jupyter Notebook interface with a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for saving, adding, deleting, and running code. The code cell contains the following Python code:

```
In [*]: 1 value = int(input("Enter a positive integer value: "))
2
3 while value != 0:
4     print(value)
5     value = value - 2
6
7 print("Exit loop")
8
```

The output of the code cell shows the prompt "Enter a positive integer value: 9" followed by a list of values: 9, 7, 5, 3, 1, -1, -3, -5, -7, -9, -11, and -13. The values 9, 7, 5, 3, and 1 are highlighted with a red box, indicating the loop is still running. The values -1, -3, -5, -7, -9, -11, and -13 are also highlighted with a red box, indicating the loop has continued to negative values, which is not the intended behavior.



How to manage a 'dead' kernel?

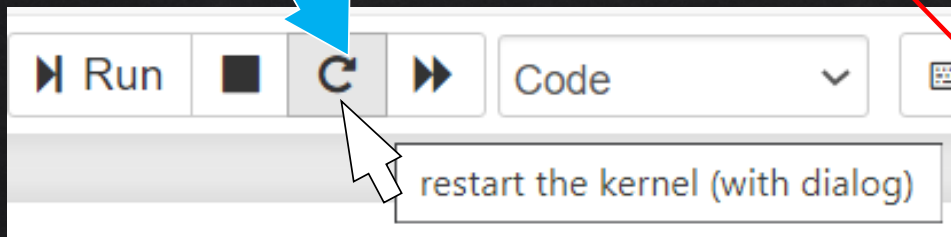
Infinite loop

```
value = 9
```

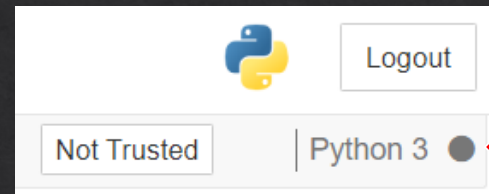
```
while value > 0:  
    print(value)  
    value = value - 2
```

```
print("Exit loop")
```

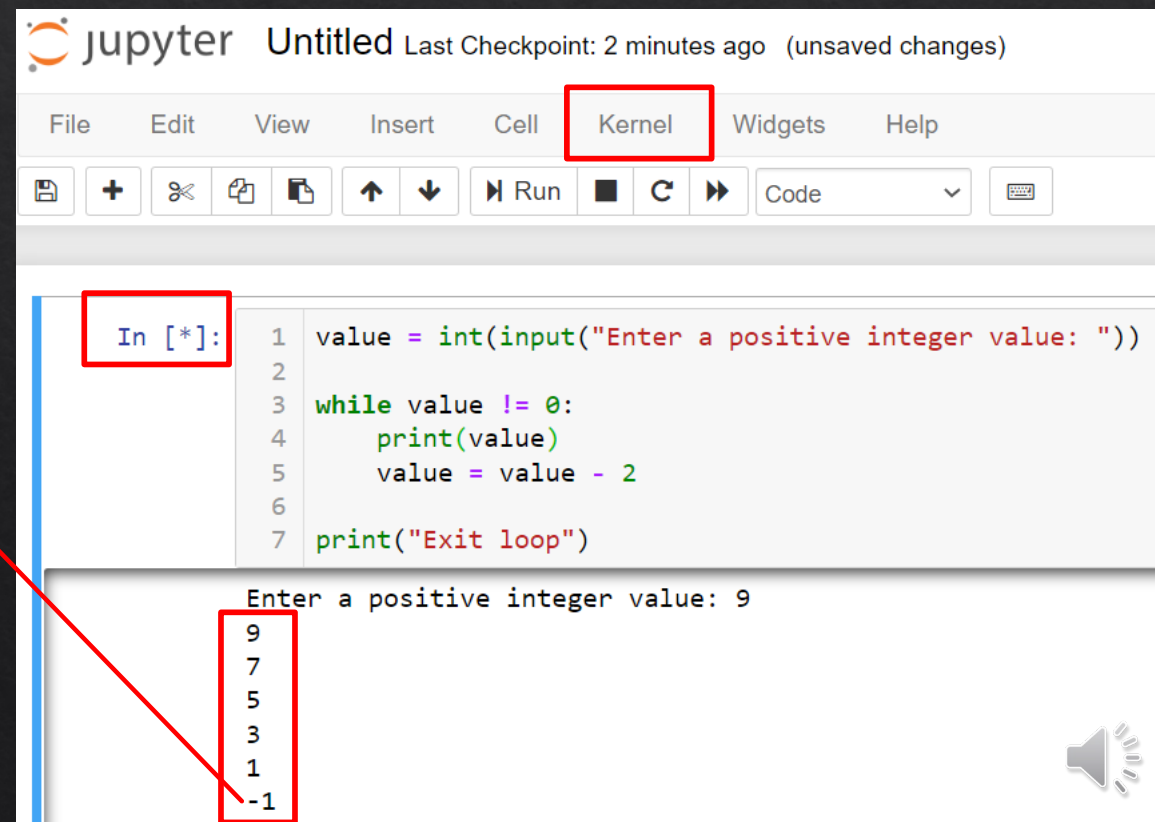
Restart the
Kernel.



observe print output



Kernel is dead.



factorial function with while-loop

Write a function `factorial(n)` that uses a while loop to compute the result.

Recall:

$$\text{factorial}(6) = 6! = 1 * 2 * 3 * 4 * 5 * 6$$

Strategy:

Use the while-loop to loop the numbers to multiply with
`factorial(6)` will loop from 1, 2, 3, ... until 6. Multiply.

Executing the strategy:

A. Setup the variables before starting the while-loop:

result: store current total counter: goes from 1 to 6

B. The code in the while loop

Multiply result with counter. Store it back to result.

Increase counter by 1



factorial function with while-loop

Write a function `factorial(n)` that computes the value for $n!$

n is 6

`factorial(6) = 6! = 1 * 2 * 3 * 4 * 5 * 6`

Initialise: `result = 1`
`counter = 1` (increase after each iteration)

1st iteration: `result = result * counter` = `1 * 1 = 1`

2nd iteration: `result = result * counter` = `1 * 2 = 2`

3rd iteration: `result = result * counter` = `2 * 3 = 6`

...

...

6th iteration: `result = result * counter` = `120 * 6 = 720`



factorial function with while-loop

- ◆ Plan: Loop and multiply. From 1, 2, 3, ... until n
- ◆ Use `result` to store the current result/total.
Use `counter` to multiply with the result.
- ◆ Know when to stop the while-loop. To get the correct result.

```
def factorial(n):  
    result = 1  
    counter = 1  
    # while counter is less than or equal to ____  
        # update result by ____  
        # update counter by ____  
    return result
```



Write the factorial function

```
def factorial(n):  
    result = 1  
    counter = 1  
    while counter <= n:  
        result = result * counter  
        counter = counter + 1  
    return result
```

result	counter
1	1
1	2
2	3
6	4
24	5
120	6
720	7

return 720

```
>>> factorial(6)
```

```
720
```

